Circuito para medir conductancia cutánea

https://gemini.google.com/app/3b24c31918474733

User prompt: dame un circuito pequeño para medir una propiedad fisiologica de un humano

Response: Un circuito de **Respuesta Galvánica de la Piel (GSR)** o **Actividad Electrodérmica (EDA)** es una de las opciones más accesibles y seguras. Mide los cambios en la conductancia eléctrica de la piel provocados por la actividad del sistema nervioso simpático (sudoración microscópica ante estrés, emoción o atención).

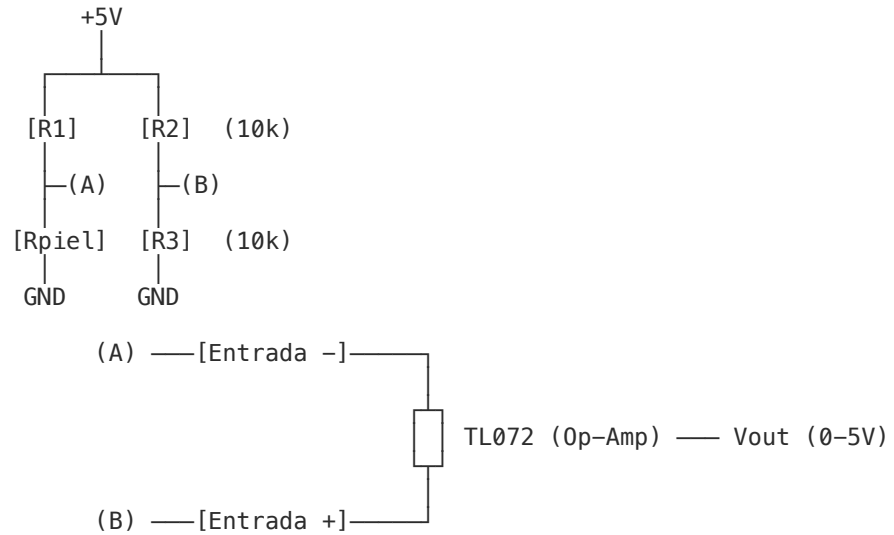
Funciona mediante un **puente de Wheatstone simple** acoplado a un amplificador operacional para acondicionar la señal.

Componentes necesarios

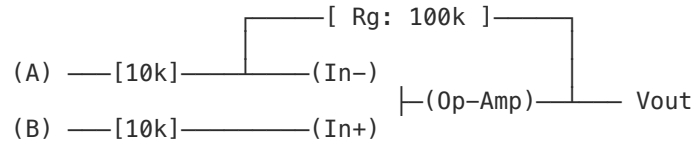
- **1x Op-Amp:** TL072 o LM358 (alimentado a $5\ \mathrm{V}$).
- **Resistencias:**
  - $R_1, R_2, R_3 = 10\ \mathrm{k\Omega}$ (1/4W)
  - $R_g(\text{ganancia}) = 100\ \mathrm{k\Omega}$ (o potenciómetro de $100\ \mathrm{k\Omega}$ para ajustar sensibilidad)
- **Electrodos:** 2 placas de cobre pequeñas o tiras de papel aluminio sujetas a los dedos índice y medio.
- **Fuente:** $5\ \mathrm{V}$ DC (regulado, por ejemplo desde un Arduino o batería).

Esquema del Circuito

El circuito utiliza una configuración de amplificador diferencial básico para comparar la resistencia de la piel (R_{piel}) contra un valor de referencia.



Para implementar la ganancia entre las entradas diferencialmente con un solo canal del TL072:



Principio de Funcionamiento

1. **Divisor de Tensión:** La piel seca actúa como una resistencia alta ($\sim 100\ \mathrm{k\Omega} - 1\ \mathrm{M\Omega}$). Al aumentar la sudoración por excitación autonómica, R_{piel} disminuye.
2. **Amplificación:** La diferencia de potencial entre los nodos (A) y (B) es pequeña. El amplificador operacional diferencial multiplica esta diferencia según la relación $A_v = \frac{R_g}{R_{\text{in}}}$.

3. **Lectura:** La salida V_{out} oscila linealmente entre $0\ \text{V}$ y $5\ \text{V}$, ideal para conectar a un pin analógico (ADC) de un microcontrolador o graficar en un osciloscopio.

Consideraciones de Seguridad

- **Aislamiento:** Si conectas este circuito a una computadora mediante USB (vía Arduino o DAQ), asegúrate de alimentar el circuito únicamente con baterías de $9\ \text{V}$ (usando un regulador 7805) o usar un aislador optoacoplador para desacoplar completamente la tierra de la red eléctrica.

User prompt: Proporciona el código de procesamiento en tiempo real para filtrar la señal GSR medida con un microcontrolador o Python.

Response: Para procesar una señal de Respuesta Galvánica de la Piel (GSR/EDA) en tiempo real se deben considerar dos componentes principales:

1. **Componente Tónica (SCL - *Skin Conductance Level*):** Variaciones lentas y continuas de baja frecuencia ($< 0.05\ \text{Hz}$).
2. **Componente Fásica (SCR - *Skin Conductance Response*):** Respuestas rápidas ante estímulos específicos, compuestas por picos que ocurren entre $0.5\ \text{Hz}$ y $2\ \text{Hz}$.

El ruido de alta frecuencia (como la interferencia de la red eléctrica a $50/60\ \text{Hz}$ o el temblor muscular) se elimina con un **filtro paso bajas (Butterworth de 2.º orden)** con frecuencia de corte alrededor de $2\ \text{Hz}$–$5\ \text{Hz}$.

Opción 1: Procesamiento en Microcontrolador (C++ / Arduino)

Para filtrar en tiempo real dentro de un microcontrolador sin consumir muchos recursos, se implementa una ecuación en diferencias finitas derivada de un filtro paso bajas IIR (Infinite Impulse Response) de 1.er o 2.º orden.

A continuación se presenta una implementación limpia utilizando un filtro de media móvil exponencial (EMA) como paso bajas ligero y la derivada discreta para detectar respuestas fásicas en tiempo real:

```
// Mapeo de pines y parámetros de tiempo
const int GSR_PIN = A0;
const float SAMPLING_FREQ = 50.0; // Frecuencia de muestreo: 50 Hz (cada 20 ms)
const unsigned long SAMPLE_INTERVAL_MS = 1000 / SAMPLING_FREQ;

// Parámetros del filtro
// alpha = dt / (RC + dt). Para fc ≈ 2 Hz a 50 Hz de muestreo, alpha ≈ 0.2
const float ALPHA = 0.2;

float filteredSignal = 0.0;
float scl = 0.0; // Nivel tónico (media muy lenta)
const float ALPHA_SCL = 0.001; // Constante de tiempo lenta (~0.01 Hz)

unsigned long lastSampleTime = 0;

void setup() {
  Serial.begin(115200);
  pinMode(GSR_PIN, INPUT);

  // Inicialización de señal con la primera lectura
  int rawADC = analogRead(GSR_PIN);
  filteredSignal = rawADC;
  scl = rawADC;
}

void loop() {
  unsigned long currentTime = millis();

  if (currentTime - lastSampleTime >= SAMPLE_INTERVAL_MS) {
    lastSampleTime = currentTime;

    // 1. Lectura analógica (0 - 1023)
    int rawValue = analogRead(GSR_PIN);
```

```
    // 2. Filtro Paso Bajas IIR (Elimina ruido > 2 Hz)
    filteredSignal = (ALPHA * rawValue) + ((1.0 - ALPHA) * filteredSignal);

    // 3. Extracción de la componente Tónica (SCL) mediante filtro de respuesta muy lenta
    scl = (ALPHA_SCL * filteredSignal) + ((1.0 - ALPHA_SCL) * scl);

    // 4. Extracción de la componente Fásica (SCR) = Señal Filtrada - Tónica
    float scr = filteredSignal - scl;

    // Salida por puerto serie (formato legible por Serial Plotter)
    Serial.print("Raw:");
    Serial.print(rawValue);
    Serial.print(",");
    Serial.print("Filtered:");
    Serial.print(filteredSignal);
    Serial.print(",");
    Serial.print("Phasic_SCR:");
    Serial.println(scr);
  }
}
```

Opción 2: Procesamiento en Python en Tiempo Real (NumPy / SciPy)

Si la señal se transmite por puerto serie (UART) a una computadora, se puede aplicar un filtro digital digital IIR **Butterworth** utilizando una estructura de búfer circular.

```
import numpy as np
import serial
from scipy.signal import butter, lfilter_zi, lfilter

class RealTimeGSRFilter:
    def __init__(self, fs=50.0, lowcut=2.0, order=2):
        """
        Filtro paso bajas Butterworth en tiempo real para señales GSR.
        :param fs: Frecuencia de muestreo en Hz.
        :param lowcut: Frecuencia de corte superior en Hz.
        :param order: Orden del filtro.
        """
        self.fs = fs
        nyq = 0.5 * fs
        normal_cutoff = lowcut / nyq

        # Coeficientes del filtro Butterworth
        self.b, self.a = butter(order, normal_cutoff, btype='low', analog=False)

        # Estado interno del filtro IIR (condiciones iniciales)
        self.zi = lfilter_zi(self.b, self.a)

        # Filtro de componente tónica (SCL)
        self.scl_alpha = 0.001
        self.scl_state = None

    def process_sample(self, raw_sample: float):
        """
        Procesa una sola muestra y retorna (señal_filtrada, fásica_scr, tónica_scl)
        """
        # Aplicar filtro IIR en tiempo real preservando el estado
        filtered_sample, self.zi = lfilter(self.b, self.a, [raw_sample], zi=self.zi)
        y = filtered_sample[0]

        # Inicialización del nivel tónico
        if self.scl_state is None:
            self.scl_state = y

        # Actualización de SCL (filtro paso bajas muy lento)
        self.scl_state = (self.scl_alpha * y) + ((1.0 - self.scl_alpha) * self.scl_state)
```

```
        # Componente Fásica (SCR)
        scr = y - self.scl_state

        return y, scr, self.scl_state


# Ejemplo de loop de transmisión por puerto serie
if __name__ == "__main__":
    # Configurar puerto serie segun corresponda (ej. '/dev/ttyUSB0' o 'COM3')
    # ser = serial.Serial('/dev/ttyUSB0', 115200)

    gsr_filter = RealTimeGSRFilter(fs=50.0, lowcut=2.0)

    print("Procesador GSR iniciado...")

    # Simulación con muestras sintéticas
    dummy_data = np.random.normal(loc=500, scale=10, size=100)

    for raw_val in dummy_data:
        filtered, scr, scl = gsr_filter.process_sample(raw_val)
        # print(f"Raw: {raw_val:.2f} | Filtrado: {filtered:.2f} | SCR: {scr:.2f} | SCL: {scl:.2f}")
```

Puntos clave para la calibración

- **Frecuencia de Muestreo (f_s):** La señal EDA/GSR es lenta; una frecuencia entre **20 Hz y 100 Hz** es suficiente para capturar eventos fisiológicos dinámicos sin sobrecargar la memoria.
- **Detección de picos de SCR:** Un evento de SCR válido típicamente presenta un tiempo de subida de 1 s a 3 s y una amplitud mínima mayor a 0.01 μS (o el equivalente en el delta del ADC).
- **Conversión a Conductancia:** Si se requiere medir en unidades físicas (μS — microSiemens), se aplica la ley de Ohm sobre el voltaje leído: $G = \frac{1}{R_{\text{piel}}} = \frac{I}{V}$.
